

下一步

使用 Kimi API 进行多轮对话

📄 复制页面



Kimi API 与 Kimi 智能助手不同，**API 本身不具有记忆功能，它是无状态的**，这意味着，当你多次请求 API 时，Kimi 大模型并不知道你前一次请求的内容，也不会记忆任何请求的上下文信息。例如，你在前一次请求中告诉 Kimi 大模型你今年 27 岁，在下一次请求中，Kimi 大模型并不会记住你 27 岁这件事。

因此，我们需要手动维护每次请求的上下文，即 Context，把上一次请求过的内容手动加入到下一次请求中，让 Kimi 大模型能正确看到此前我们都聊了什么。我们将改造上一章节中使用的示例，来展示如何通过维护 messages 列表让 Kimi 大模型拥有记忆，并实现多轮对话功能。

注：我们将实现多轮对话的要点以注释的形式添加到代码中。

`python` `node.js`

KIMI 开放平台



```
api_key = "MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
base_url = "https://api.moonshot.cn/v1",
)
```

```
# 我们定义一个全局变量 messages，用于记录我们和 Kimi 大模型产生的历史对话消息
# 在 messages 中，既包含我们向 Kimi 大模型提出的问题 (role=user)，也包括 Kimi 大模型给我们的
# 当然，也包括初始的 System Prompt (role=system)
# messages 中的消息按时间顺序从小到大排列
```

```
messages = [
    {"role": "system", "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅"}
]
```

```
def chat(input: str) -> str:
```

```
    """
```

```
    chat 函数支持多轮对话，每次调用 chat 函数与 Kimi 大模型对话时，Kimi 大模型都会"看到"此前
    产生的历史对话消息，换句话说，Kimi 大模型拥有了记忆。
    """
```

```
global messages
```

```
# 我们将用户最新的问题构造成一个 message (role=user)，并添加到 messages 的尾部
```

```
messages.append({
    "role": "user",
    "content": input,
})
```

```
# 携带 messages 与 Kimi 大模型对话
```

```
completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=messages
)
```

```
# 通过 API 我们获得了 Kimi 大模型给予我们的回复消息 (role=assistant)
```

```
assistant_message = completion.choices[0].message
```

```
# 为了让 Kimi 大模型拥有完整的记忆，我们必须将 Kimi 大模型返回给我们的消息也添加到 messages
messages.append(assistant_message)
```

KIMI 开放平台



```
print(chat("你好，我今年 27 岁。"))  
print(chat("你知道我今年几岁吗？")) # 在这里，Kimi 大模型根据此前的上下文信息，将会知道你今年
```

我们回顾一下上述代码中的要点：

- Kimi API 本身没有上下文记忆功能，我们需要通过 API 中的 `messages` 参数，手动将“之前聊了什么”告知 Kimi 大模型；
- 在 `messages` 中，既要存储我们向 Kimi 大模型提出的问题消息（`role=user`），也要存储 Kimi 大模型给我们的回复消息（`role=assistant`）；

需要注意的是，在上述代码中，随着 `chat` 调用次数的不断增多，`messages` 列表的长度也在不断增加，这意味着每次请求所消耗的 Tokens 数量也在不断增加，并且最终会在某个时间点，`messages` 中的消息所占用的 Tokens 超过了 Kimi 大模型支持的上下文窗口大小。我们推荐你使用某种策略来保持 `messages` 列表的消息数量在一个可控的范围内，例如，每次只保留最新的 20 条消息作为本次请求的上下文。

我们给出一个示例，以便于你能顺利理解如何控制上下文长度，请注意关注 `make_messages` 函数是如何运作的：

`python` `node.js`

KIMI 开放平台



```
api_key = "MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
base_url = "https://api.moonshot.cn/v1",
)
```

```
# 我们将 System Messages 单独放置在一个列表中，这是因为每次请求都应该携带 System Messages
system_messages = [
    {"role": "system", "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅
]
```

```
# 我们定义一个全局变量 messages，用于记录我们和 Kimi 大模型产生的历史对话消息
# 在 messages 中，既包含我们向 Kimi 大模型提出的问题 (role=user)，也包括 Kimi 大模型给我们的
# messages 中的消息按时间顺序从小到大排列
messages = []
```

```
def make_messages(input: str, n: int = 20) -> list[dict]:
```

```
    """
```

使用 make_messages 控制每次请求的消息数量，使其保持在一个合理的范围内，例如默认值是 20。在的过程中，我们会先添加 System Prompt，这是因为无论如何对消息进行截断，System Prompt 都是的内容，再获取 messages — 即历史记录中，最新的 n 条消息作为请求使用的消息，在大部分场景中能保证请求的消息所占用的 Tokens 数量不超过模型上下文窗口。

```
    """
```

```
    global messages
```

```
# 首先，我们将用户最新的问题构造成一个 message (role=user)，并添加到 messages 的尾部
messages.append({
    "role": "user",
    "content": input,
})
```

```
# new_messages 是我们下一次请求使用的消息列表，现在让我们来构建它
new_messages = []
```

```
# 每次请求都需要携带 System Messages，因此我们需要先把 system_messages 添加到消息列表中
# 注意，即使对消息进行截断，也应该注意保证 System Messages 仍然在 messages 列表中。
new_messages.extend(system_messages)
```

KIMI

开放平台



```
messages = messages[-n:]
```

```
new_messages.extend(messages)
```

```
return new_messages
```

```
def chat(input: str) -> str:
```

```
    """
```

chat 函数支持多轮对话，每次调用 chat 函数与 Kimi 大模型对话时，Kimi 大模型都会"看到"此前产生的历史对话消息，换句话说，Kimi 大模型拥有了记忆。

```
    """
```

```
# 携带 messages 与 Kimi 大模型对话
```

```
completion = client.chat.completions.create(
```

```
    model="kimi-k2.6",
```

```
    messages=make_messages(input)
```

```
)
```

```
# 通过 API 我们获得了 Kimi 大模型给予我们的回复消息 (role=assistant)
```

```
assistant_message = completion.choices[0].message
```

```
# 为了让 Kimi 大模型拥有完整的记忆，我们必须将 Kimi 大模型返回给我们的消息也添加到 messages
```

```
messages.append(assistant_message)
```

```
return assistant_message.content
```

```
print(chat("你好，我今年 27 岁。"))
```

```
print(chat("你知道我今年几岁吗？")) # 在这里，Kimi 大模型根据此前的上下文信息，将会知道你今年
```

请注意，上述的代码示例仅考虑了最简单的调用场景，在实际的业务代码逻辑中，你可能需要考虑更多的场景和边界，例如：

- 并发场景下可能需要额外的读写锁；
- 针对多用户场景，需要为每个用户单独维护 messages 列表；
- 你可能需要对 messages 列表进行持久化；

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

- 你可能想对被遗弃的消息做一次总结，并生成一条新的消息加入到 messages 列表中；

KIMI 开放平台



>

此页面对您有帮助吗？

 是

 否

< 使用思考模型

视觉模型 >

